

SMART +
SUSTAINABLE

Liderar la construcción **de un mundo sostenible**

Aprendizaje Supervisado

Bastián Galasso

Data Analytics Lead – Coca-Cola Embonor

Julio 2025

Aprendizaje Supervisado

01 K-Nearest Neighbors

02 Naive Bayes

03 Support Vector Machines (SVM)

04 Classification Trees

00 Introducción

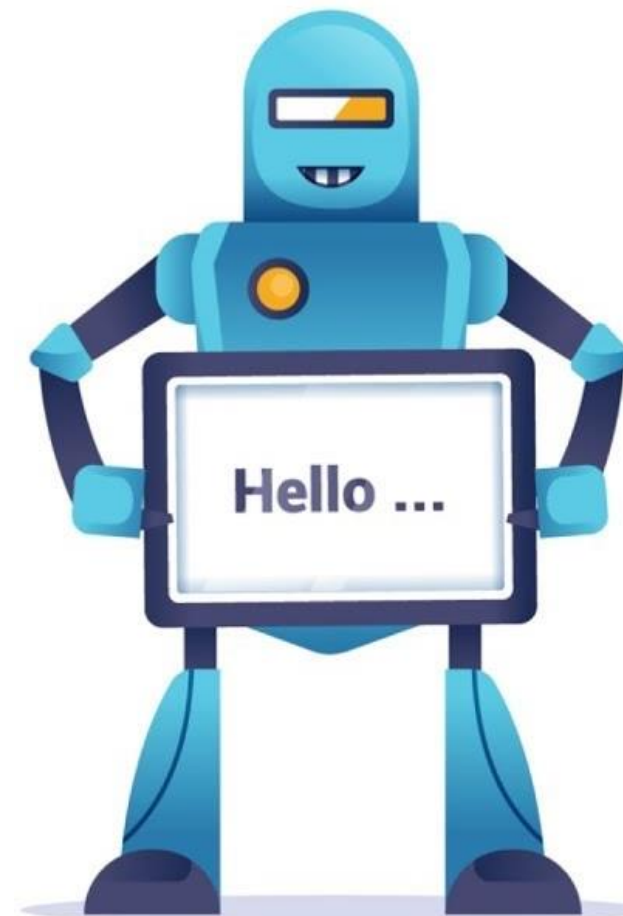
- Recordemos... ¿Qué son los algoritmos supervisado?
- En los algoritmos de aprendizaje supervisado la data es de la forma $\{x_i, y_i\}_{i=1}^n$ donde $X = \{x_i\}_{i=1}^n$ se denominan inputs o variables de entrada (predictores), e $Y = \{y_i\}_{i=1}^n$ es el output o variable respuesta (target).
- El modelo de aprendizaje supervisado representa la función $f(X) = Y$ que mapea los inputs en los outputs



- Dependiendo de la naturaleza de la variable respuesta Y , el modelo será de clasificación (si Y es categórica) o de regresión (si Y es numérica), por lo que es importante reconocer la naturaleza de la variable respuesta para poder escoger el modelo adecuado.
- Para esta clase, nos enfocaremos en aquellos modelos donde la variable respuesta Y es categórica

00 Introducción

- ¿De dónde viene el nombre supervisado?
- La respuesta es de la variable respuesta, esto porque con ella podemos **medir y supervisar** el aprendizaje del modelo, para poder ir guiando su entrenamiento acorde a ello.
- Esto se hace básicamente comparando las predicciones del modelo (variable respuesta predicha) con los valores reales (que por la naturaleza de los datos, es conocida!), permitiendo así medir el desempeño del modelo e ir haciendo las modificaciones necesarias, con el fin de obtener un mejor modelo.



00 Introducción

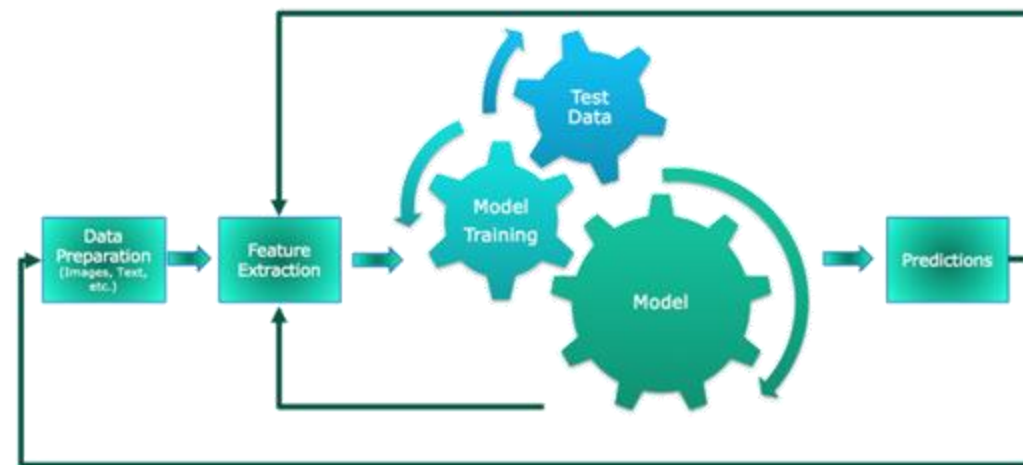
→ Machine Learning Pipeline

→ Cuando estamos en un contexto de modelos supervisados debemos saber que existe un procedimiento, medianamente estandarizado, que debemos seguir para poder desarrollar un modelo de machine learning de manera exitosa.

→ El proceso es como sigue:

- Preparación de datos
 - Limpieza y formateo de datos
 - Feature selection, extraction and engineering
 - Train y test split
 - Balanceo de clases

- Entrenamiento del modelo
- Optimización
 - Optimización de hiperparámetros
- Evaluación del modelo
- Comunicación de resultados



→ **Train/Test Split – Validación cruzada**

→ Existen distintas divisiones que se realizan en el dataset con distintos fines

- **Train/test Split:** Sirven para entrenar y evaluar el modelos
- **K-folds:** Sirven para realizar validación cruzada, que es utilizada para optimizar modelos o para validarlos.



→ **Métricas de desempeño**

- Para evaluar nuestros modelos, existen diversas métricas para hacerlo, que se puede ocupar de manera individual o conjunta, dependiendo de cada caso.

Modelos de Clasificación

- ✓ Accuracy
- ✓ Recall
- ✓ Precision
- ✓ F1 Score
- ✓ ROC - AUC

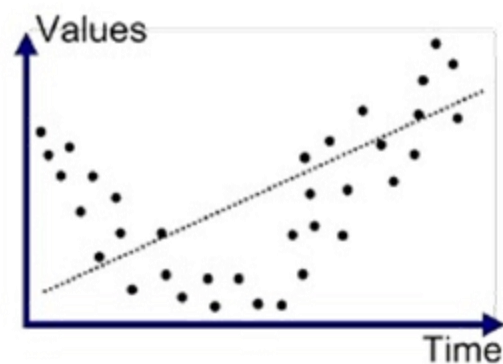
Modelos de Regresión

- ✓ MAE
- ✓ RMSE
- ✓ MSE
- ✓ R squared
- ✓ MAPE

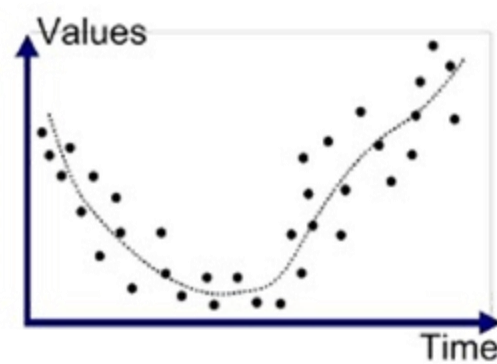
00 Introducción

→ Under/Over - fitting

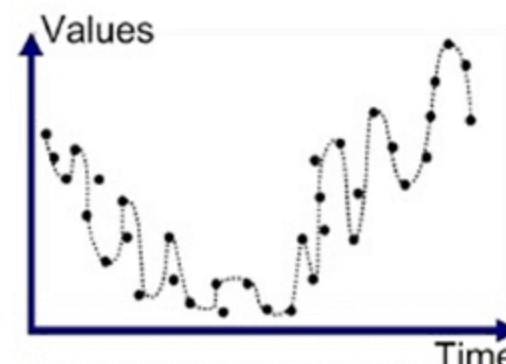
- Este es uno de los problemas bien clásicos a los que uno puede enfrentarse al momento de entrenar modelos de machine learning



Underfitted



Good Fit/Robust

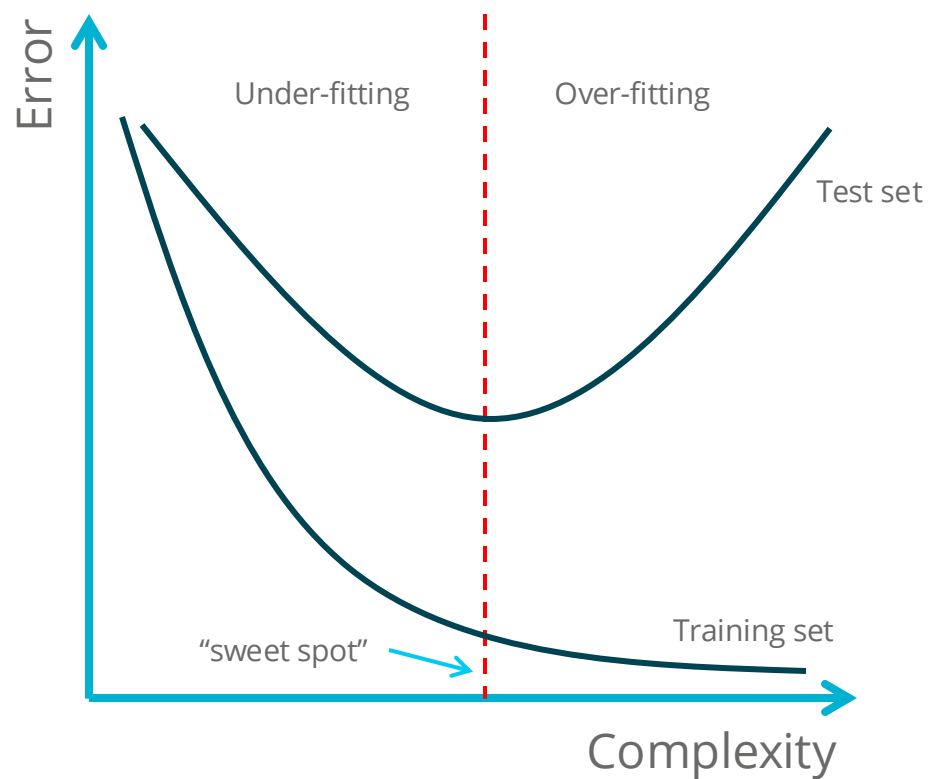


Overfitted

00 Introducción

→ Under/Over - fitting

→ ¿Cómo determinarlo?

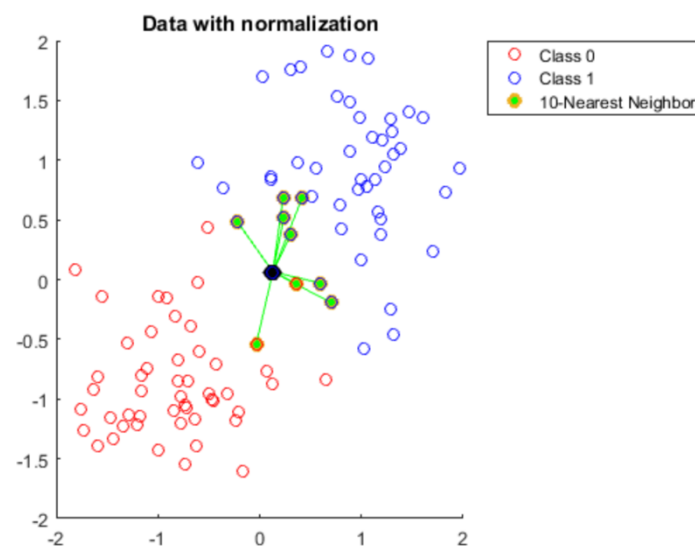
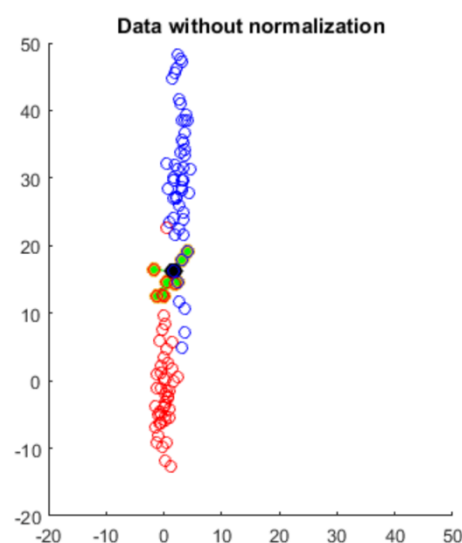


K-Nearest Neighbors

01

01 K-Nearest Neighbors

- K-nearest neighbors o K-vecinos más cercanos es una técnica de clasificación supervisada que se basa en calcular las K observaciones más cercanas según algún tipo de distancia (K vecinos), y asume la clasificación dependiendo de estos vecinos. En este sentido, no se construye un modelo global, si no que uno local.
- Es un modelo completamente basado en distancias, por lo tanto puede ser necesario estandarizar las variables para obtener buenos resultados. Por otro lado, el hecho de estar basado en distancias hace que sea un modelo simple, pero más costoso computacionalmente.



01 | K-Nearest Neighbors

¿Valor óptimo de K?

Se busca K tal que acierte en un mayor porcentaje.

En general:

$K=1$: si hay ruido, producen mucha influencia.

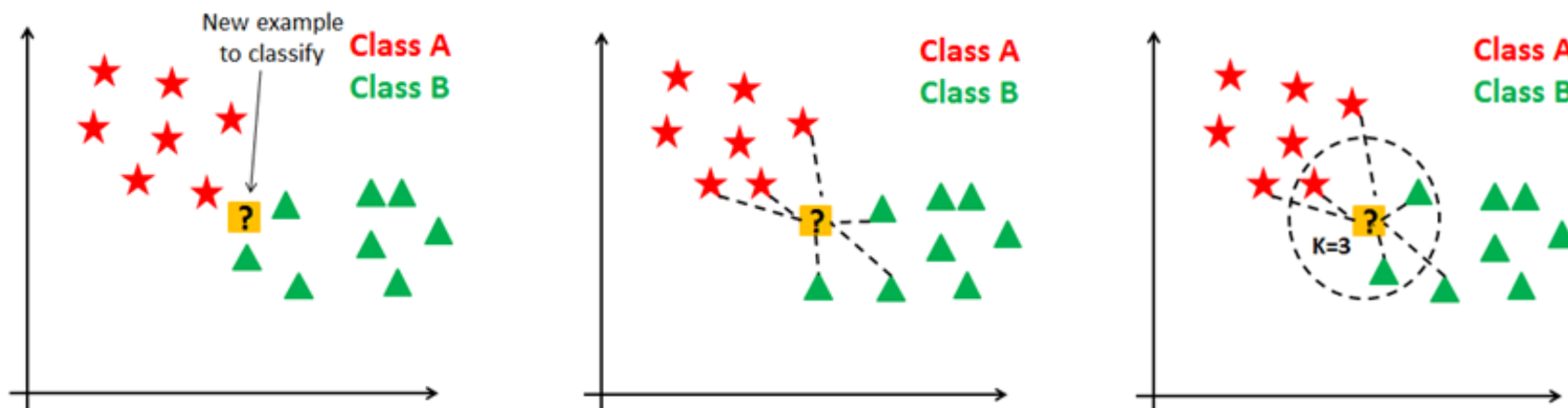
$K>1$: poca influencia de ruido.

K muy grande: se pierde la idea de localidad.



01 K-Nearest Neighbors

Ejemplo



01 | K-Nearest Neighbors

→ Dado que ya estamos en los modelos supervisados, entonces es necesario conocer algunos de los hiper-parámetros que podemos modificar, con el fin de optimizar el modelo en términos de desempeño. Estos parámetros son:

- **N_neighbors:** Es el número de vecinos que será considerados en el modelo.
- **Weights:** Función de pesos que se utilizar de los vecinos según distancia. Posibles valores son:
 - **Uniform:** Todos los puntos tienen el mismo peso
 - **Distance:** Los pesos son inversos a su distancia, mientras más cerca más influyentes son en la clasificación.
- **Metric:** Es la distancia que se utilizará para realizar los cálculos. Estas puede ser alguna de las siguientes:
 - **Minkowski**
 - **Euclidean**
 - **Manhattan**
 - **L1**
 - **L2**

Existen más hiper-parámetros, los cuales pueden encontrar en el siguiente link:

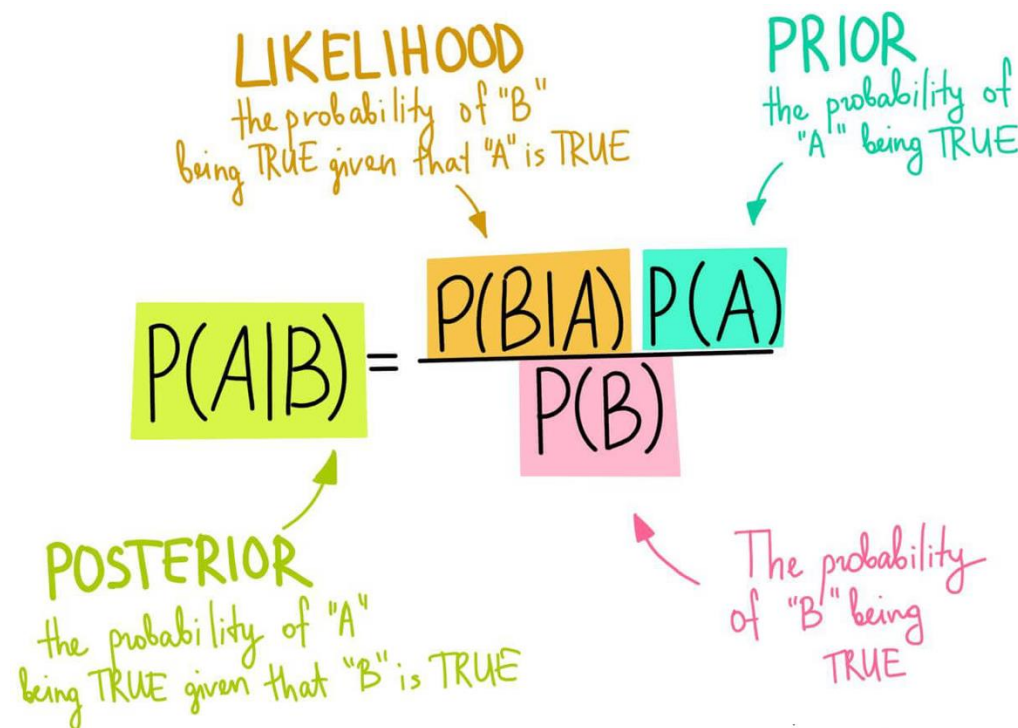
<https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsClassifier.html>

Naive Bayes

02

02 Naive Bayes

- Naive Bayes o Bayes ingenuo corresponde a una técnica de machine learning supervisado, que es conocido como un clasificador estadístico, dado que se basa en el famoso Teorema de Bayes, que predice la probabilidad de pertenecer a las distintas clases, y luego predica la clase dependiendo de cuál es más probable.
- Se llama Bayes ingenuo pues asume independencia condicional entre los atributos del dataset, lo cual es muy poco frecuente de encontrar. Dentro de sus principales bondades esta que es fácil de construir y rápido para conjuntos de datos muy grandes.



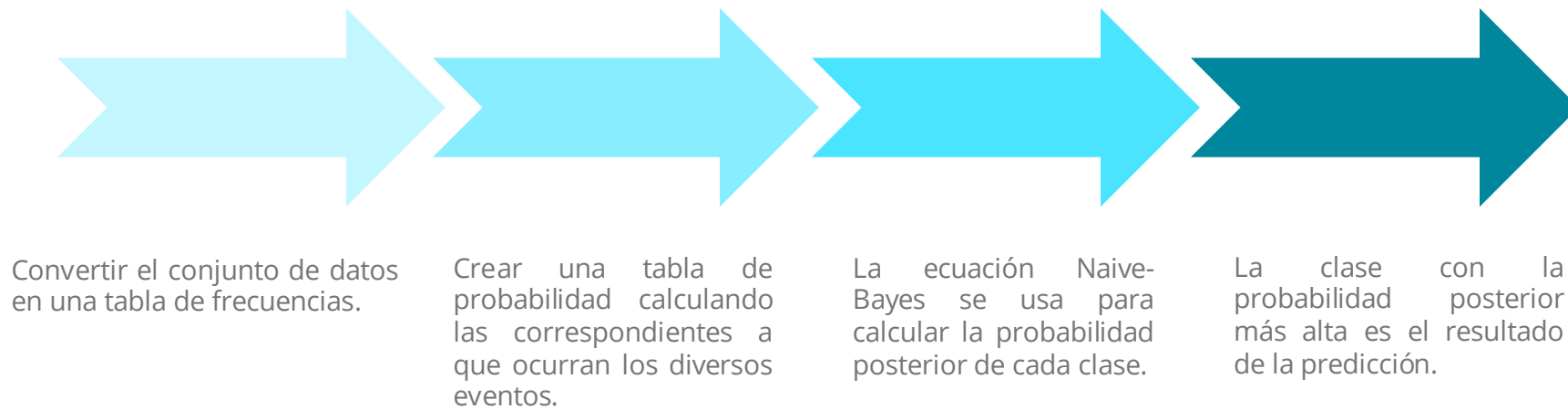
The diagram illustrates the Naive Bayes formula with color-coded components and handwritten explanations:

- LIKELIHOOD** (orange text): the probability of "B" being TRUE given that "A" is TRUE. Points to the $P(B|A)$ term (orange box).
- PRIOR** (green text): the probability of "A" being TRUE. Points to the $P(A)$ term (green box).
- POSTERIOR** (green text): the probability of "A" being TRUE given that "B" is TRUE. Points to the $P(A|B)$ term (green box).
- The probability of "B" being TRUE** (pink text): Points to the $P(B)$ term (pink box).

$$P(A|B) = \frac{P(B|A) P(A)}{P(B)}$$

02 Naive Bayes

Algoritmo



02 Naive Bayes

Ejemplo

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$



Paso 1

Weather	Play
Sunny	No
Overcast	Yes
Rainy	Yes
Sunny	Yes
Sunny	Yes
Overcast	Yes
Rainy	No
Rainy	No
Sunny	Yes
Rainy	Yes
Sunny	No
Overcast	Yes
Overcast	Yes
Rainy	No

Frequency Table		
Weather	No	Yes
Overcast		4
Rainy	3	2
Sunny	2	3
Grand Total	5	9

Likelihood table		
Weather	No	Yes
Overcast		4
Rainy	3	2
Sunny	2	3
All	5	9
	=5/14 0.36	=9/14 0.64

Paso 2

$$P[\text{Yes} | \text{Rainy}] = \frac{0.22 * 0.64}{0.36} = 0.40 \quad P[\text{No} | \text{Rainy}] = 0.60$$

$$P[\text{Yes} | \text{Overcast}] = \frac{0.44 * 0.64}{0.29} = 0.98 \quad P[\text{No} | \text{Overcast}] = 0.02$$

$$P[\text{Yes} | \text{Sunny}] = \frac{0.33 * 0.64}{0.36} = 0.59 \quad P[\text{No} | \text{Sunny}] = 0.41$$

Paso 3

02 Naive Bayes

Matemáticamente

- La idea es encontrar: $P(Y = y | X_1 = x_1, \dots, X_m = x_m)$
- Usando el teorema de Bayes y el supuesto de independencia condicional, tenemos que:

$$\begin{aligned} & P(Y = y | X_1 = x_1, \dots, X_m = x_m) \\ & \propto P(Y = y) P(X_1 = x_1, \dots, X_m = x_m | Y = y) \\ & \approx P(Y = y) \prod_{i=1}^m P(X_i = x_i | Y = y) \end{aligned}$$

Luego la predicción será la clase que maximiza la probabilidad condicional

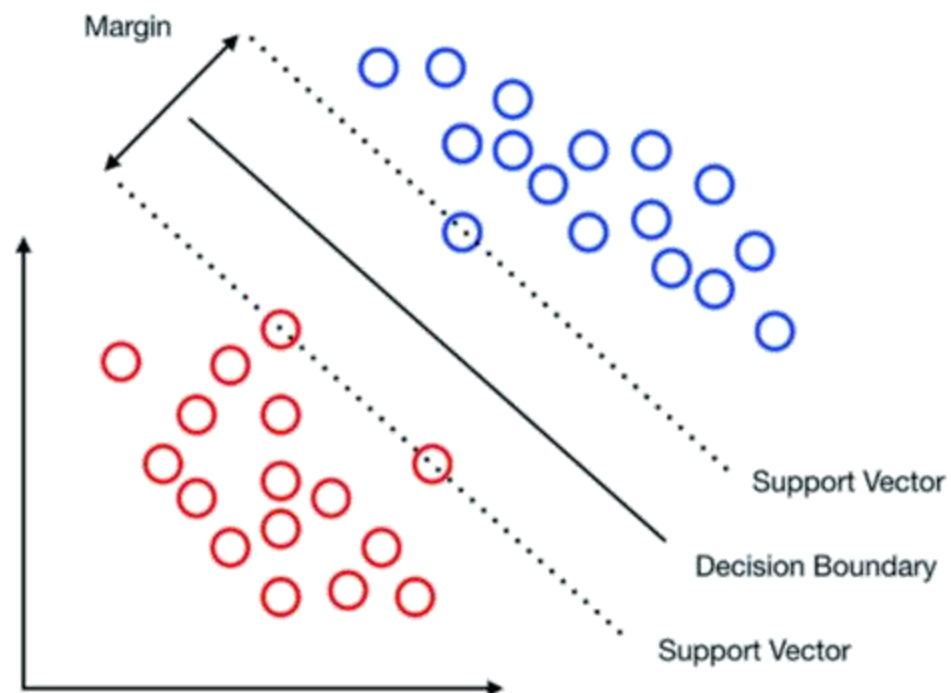
$$\hat{y}^{NB} = \underset{y}{\operatorname{argmax}} \left\{ P(Y = y) \prod_{i=1}^m P(X_i = x_{(test,i)} | Y = y) \right\}$$

Support Vector Machine

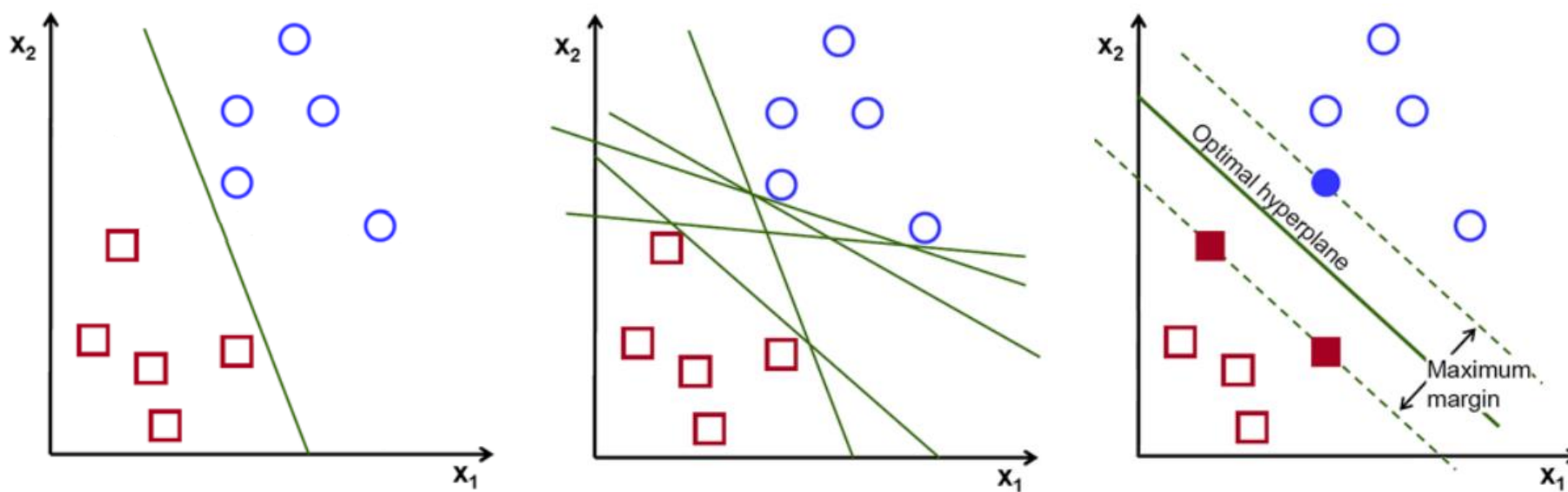
03

03 Support Vector Machine

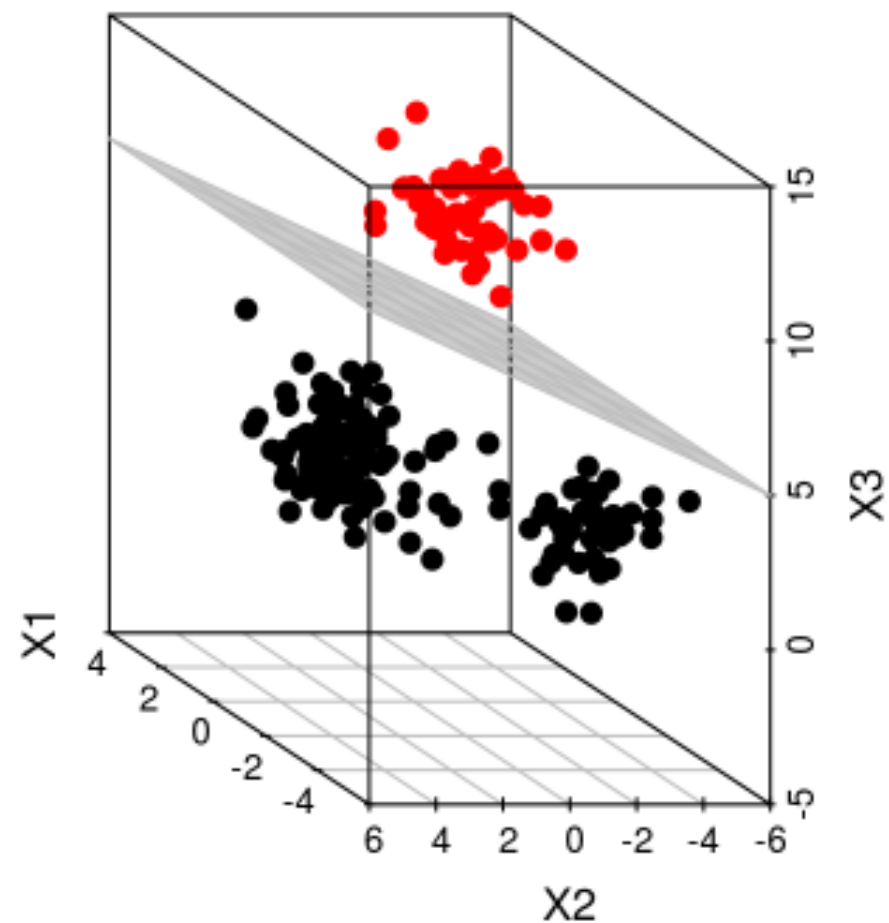
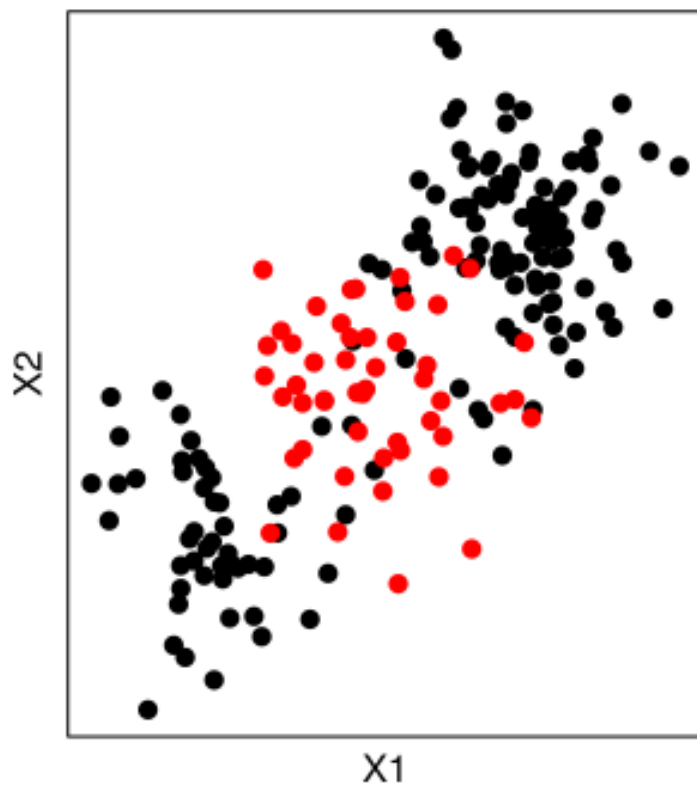
- Corresponde a una técnica de Machine Learning de tipo supervisado principalmente, que permite hacer modelos de clasificación y regresión.
- La idea principal del support vector machine es encontrar un hiper-plano (plano en más de 3 dimensiones) que separe de la mejor manera dos categorías (aquí veremos el problema de clasificación), maximizando el margen de separación entre ellas.
- El margen de separación se define como la distancia perpendicular entre el hiperplano (decisión boundary) y los registros más cercanos a cada lado.
- La posición del hiperplano es definida por un número pequeño de observaciones, los cuales se llaman vectores de soporte, y dan el nombre al método.



03 Support Vector Machine



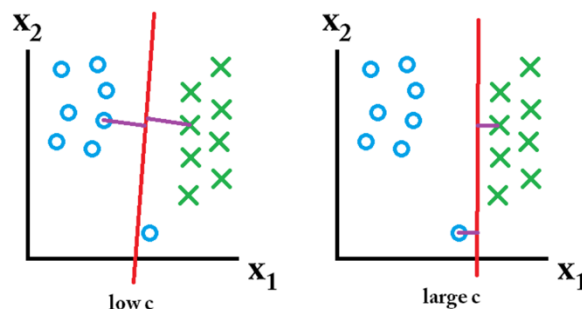
03 Support Vector Machine



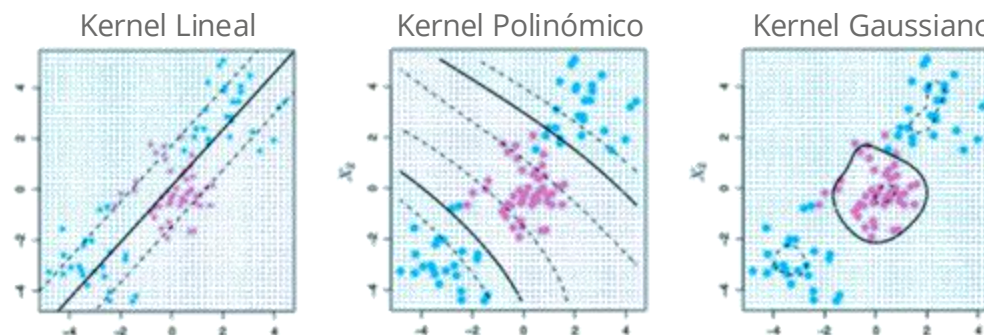
03 Support Vector Machine

Los principales hiper-parámetros que podemos manipular en el modelo SVM, son los siguientes:

- **C:** Parámetro de regularización que permite mayor o menos errores de clasificación del modelo. A mayor valor de C, es menos tolerante a fallas.



- **Kernel:** Controla la forma en que se hará la separación del espacio para poder generar la clasificación.



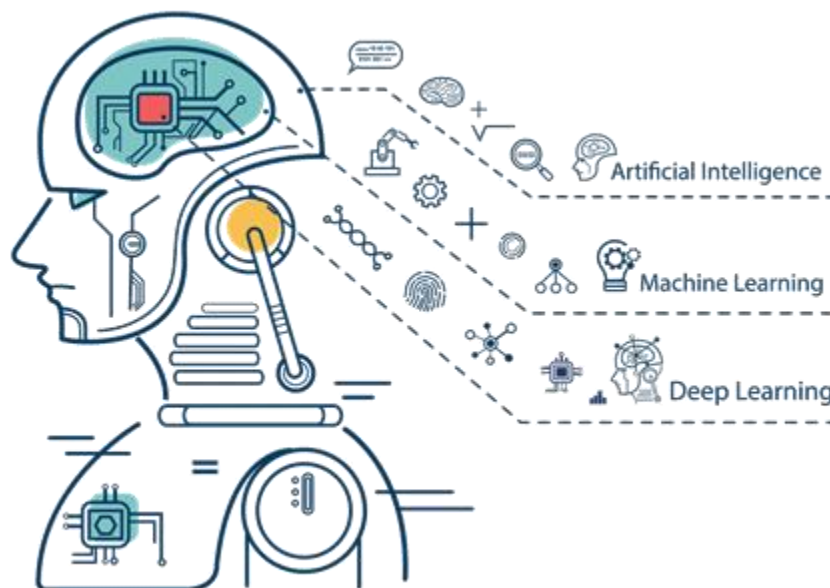
Classification Trees

04

04 Classification Trees

Corresponde a un modelo de tipo supervisado que en base a ciertos indicadores, va dividiendo en grupos a los datos en base a preguntas. En este sentido, lo que uno obtiene con un árbol de clasificación (y también de regresión) es una secuencia de preguntas a los datos, que te llevan a un resultados específico o predicción.

Dependiendo de la variable target Y , el árbol será de regresión (Y continua) o será de clasificación (Y discreta). Para esta clase nos enfocaremos solo en los de clasificación.

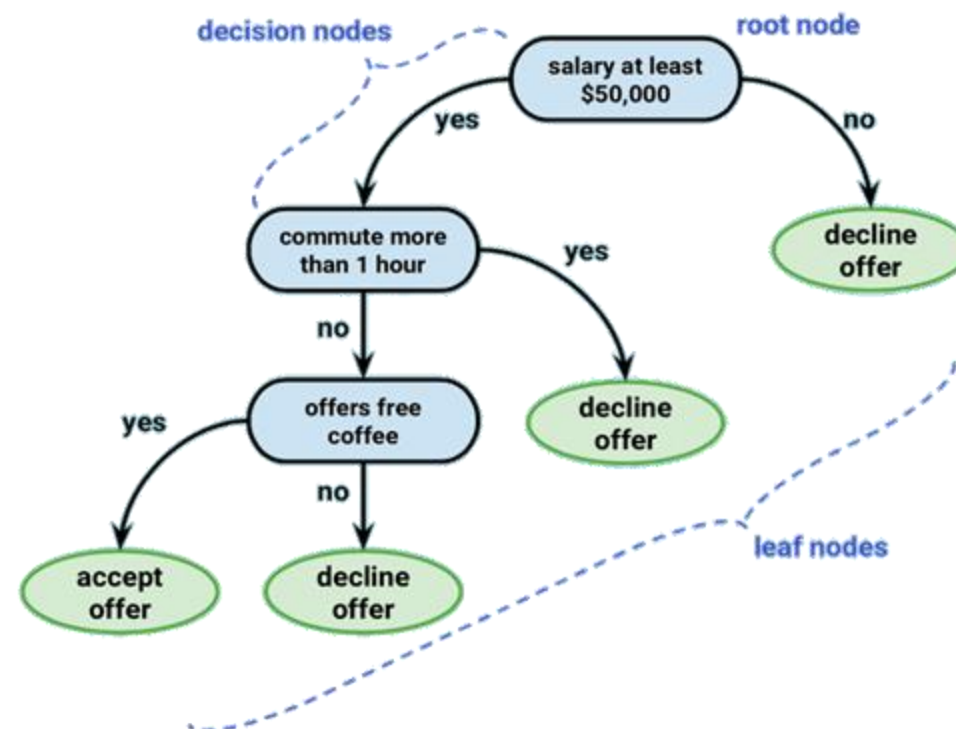


04 Classification Trees

Ejemplo de árbol

Dentro de lo que se observa en la figura, tenemos las siguientes partes:

- **Decision nodes:** representa una pregunta sobre un atributo. El nodo raíz (root node) corresponde a la primera pregunta.
- **Ramas:** Representa una posible respuesta a las preguntas.
- **Leaf nodes:** Cada hoja o nodo terminal (leaf node) representa la clase o predicción al seguir ese camino de respuestas.



04 Classification Trees

Criterios de ramificación:

Entropía o Ganancia de información

Este método trata de representar cuánta información he ganado particionando por cierto atributo y se relaciona con el grado de impureza del nodo

- Si el resultado al separar por un atributo es obvio (alta impureza del nodo 50%-50%), no he ganado información, por lo tanto no valor mucho la pena utilizar dicho atributo.
- Por otro lado, si la separación no resulta tan obvia (baja impureza 90%-10%), he ganado mucha información.

Para calcular la entropía asociada luego de una partición sugerida por un atributo A, se sigue la siguiente formula

$$Info_A(D) = \sum_{j=1}^v q_j \cdot Info(D_j)$$

donde q_j corresponde a la probabilidad de pertenecer a la categoría j del atributo A, e $Info(D_j)$ está dado por

$$Info(D) = - \sum_{i=1}^m p_i \log_2(p_i)$$

donde p_i corresponde a la probabilidad de pertenecer a la clase C_i .

04 Classification Trees

Criterios de ramificación:

Impureza de Gini

Representa la probabilidad de que una muestra seleccionada al azar de un nodo, se clasifique incorrectamente de acuerdo con la distribución de la muestra en el nodo. El objetivo es minimizar estas impurezas

Para calcular dicha impureza se realiza lo siguiente: Los datos se dividen según el atributo A , en dos regiones A_1 y A_2 , con n_1 y n_2 elementos respectivamente. p_{ji} corresponde a la proporción de elementos en A_j con $j = 1, 2$ que pertenecen a la clase i con $i = 1, \dots, m$.

Luego, la impureza de Gini $I(A_j)$ se calcula:

$$\begin{aligned} &= p_{j1}(1 - p_{j1}) + p_{j2}(1 - p_{j2}) + \dots + p_{jm}(1 - p_{jm}) \\ &= \sum_{i=1}^m p_{ji}(1 - p_{ji}) = \sum_{i=1}^m (p_{ji} - p_{ji}^2) = \sum_{i=1}^m p_{ji} - \sum_{i=1}^m p_{ji}^2 \end{aligned}$$

Y el índice de Gini se calcula

$$Gini(A) = q_1 I(A_1) + q_2 I(A_2)$$

donde q_j corresponde a la probabilidad de pertenecer a la categoría j del atributo A .

04 Classification Trees

Los principales hiper-parámetros que utilizaremos en los árboles de clasificación, son los siguientes:

- **criterion:** Respecto a cual función de pérdida se realizarán las divisiones
- **Max_depth:** Profundidad máxima del árbol
- **Ccp_Alpha:** parámetro de complejidad del árbol, que se utiliza para “podar”.

Existen más hiper-parámetros pero estos son los más utilizados. De todas formas, los demás se pueden revisar en la documentación en

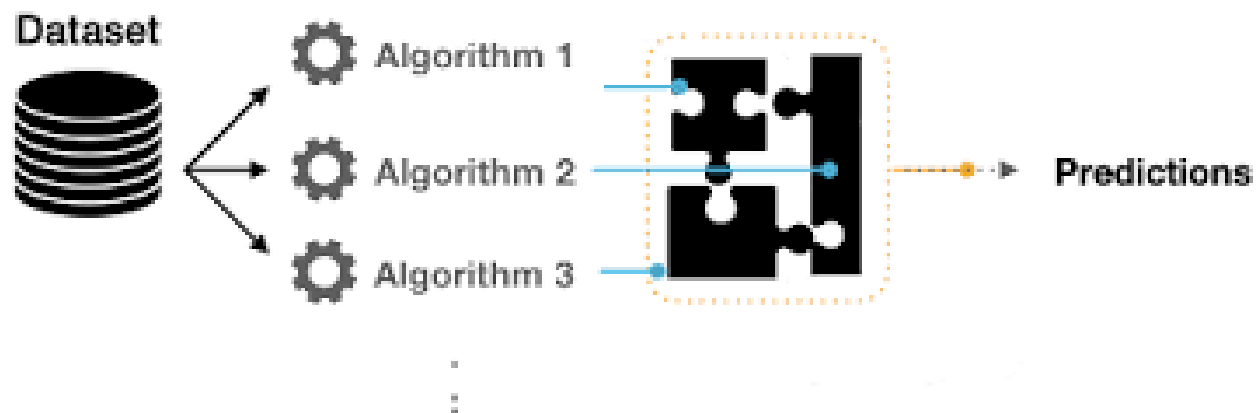
<https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html#sklearn.tree.DecisionTreeClassifier>

Random Forest

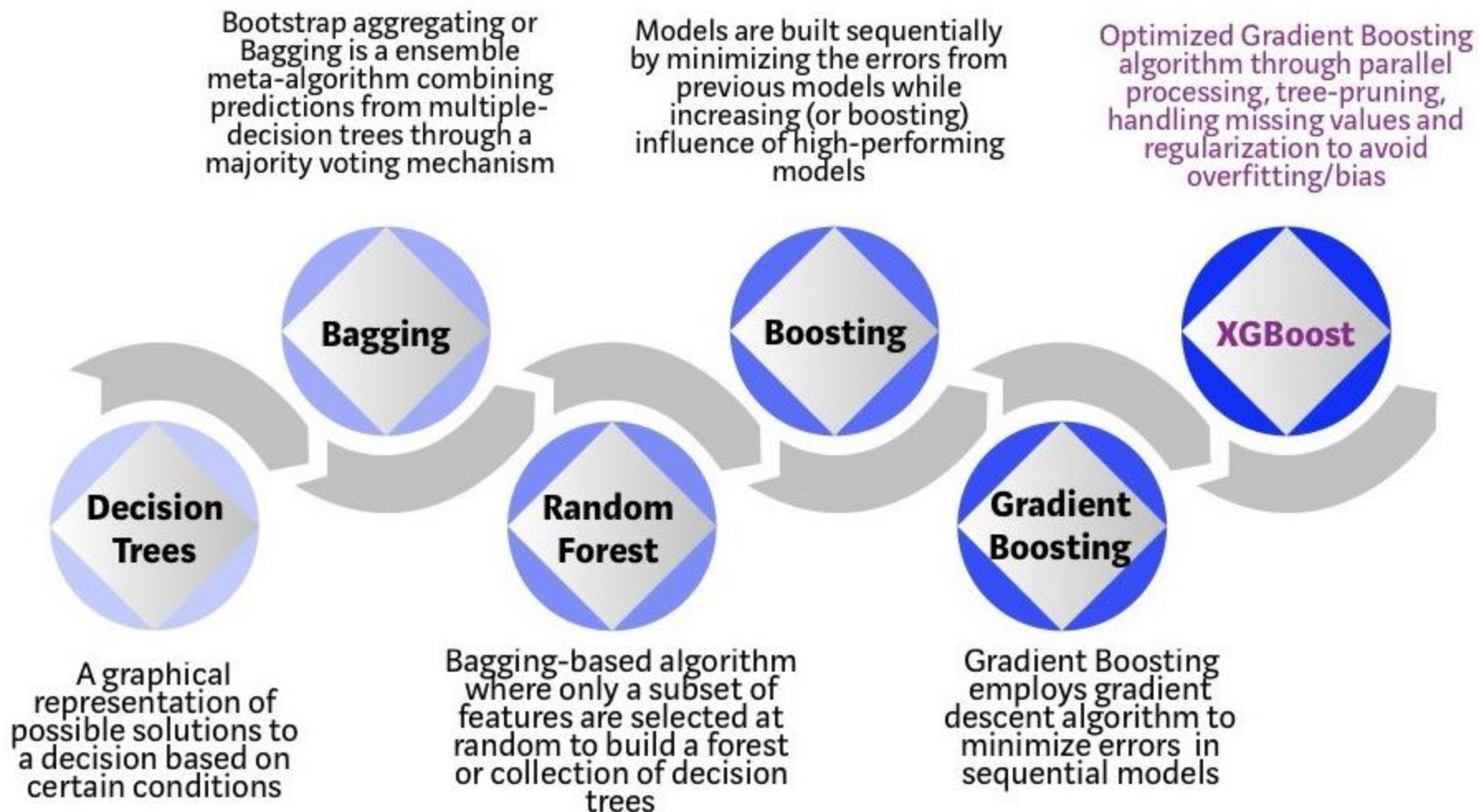
05

00 Introducción

- En estadística y machine learning, los métodos ensamblados consisten en el uso de múltiples algoritmos en orden de obtener un mejor rendimiento predictivo, del que podríamos obtener con cualquier de los algoritmos por si solos.
- Los ensambles pueden ser desde repeticiones de un mismo modelo, hasta versiones más sofisticadas con distintos modelos y ponderaciones de cada uno, permitiendo así una técnica con estructura muy flexible y útil.
- En esta clase revisaremos un grupo particular de modelos ensamblados, todos ellos basados en arboles de decisión, pero de distintas maneras y también con distintos grados de dificultad.



00 Introducción

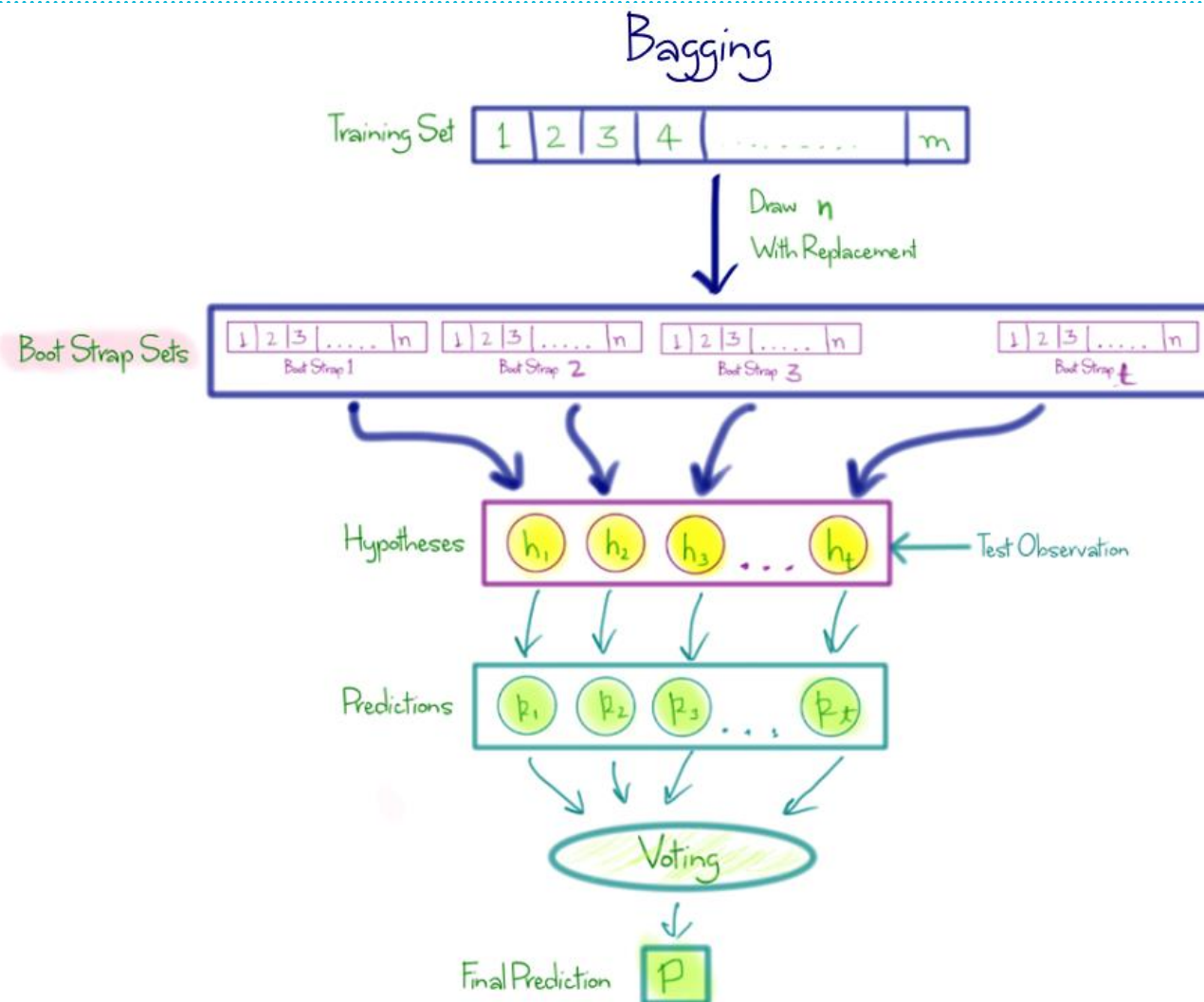


01 Random Forest y Bagging

- Bagging (Bootstrap Aggregating) es un algoritmo ensamblado diseñado para mejorar la estabilidad y precisión de los algoritmos utilizados tanto para clasificación como regresión.
- Si bien Bagging es generalmente aplicado a los métodos de árboles de decisión, la técnica es mucho más amplia y flexible por lo que se puede utilizar con cualquier tipo de algoritmo-método.



01 Random Forest y Bagging



01 Random Forest y Bagging

→ Dentro de las principales ventajas que tienen los modelos de Bagging, destacan las siguientes:

- Son modelos que funcionan muy bien cuando los datos no permiten generar un modelo estable.
- Disminuye la posibilidad de sobre-ajuste
- Mejora los pronósticos, debido a que al ajustar tantos árboles se empiezan a encontrar patrones que modelos simples no lo hacen.

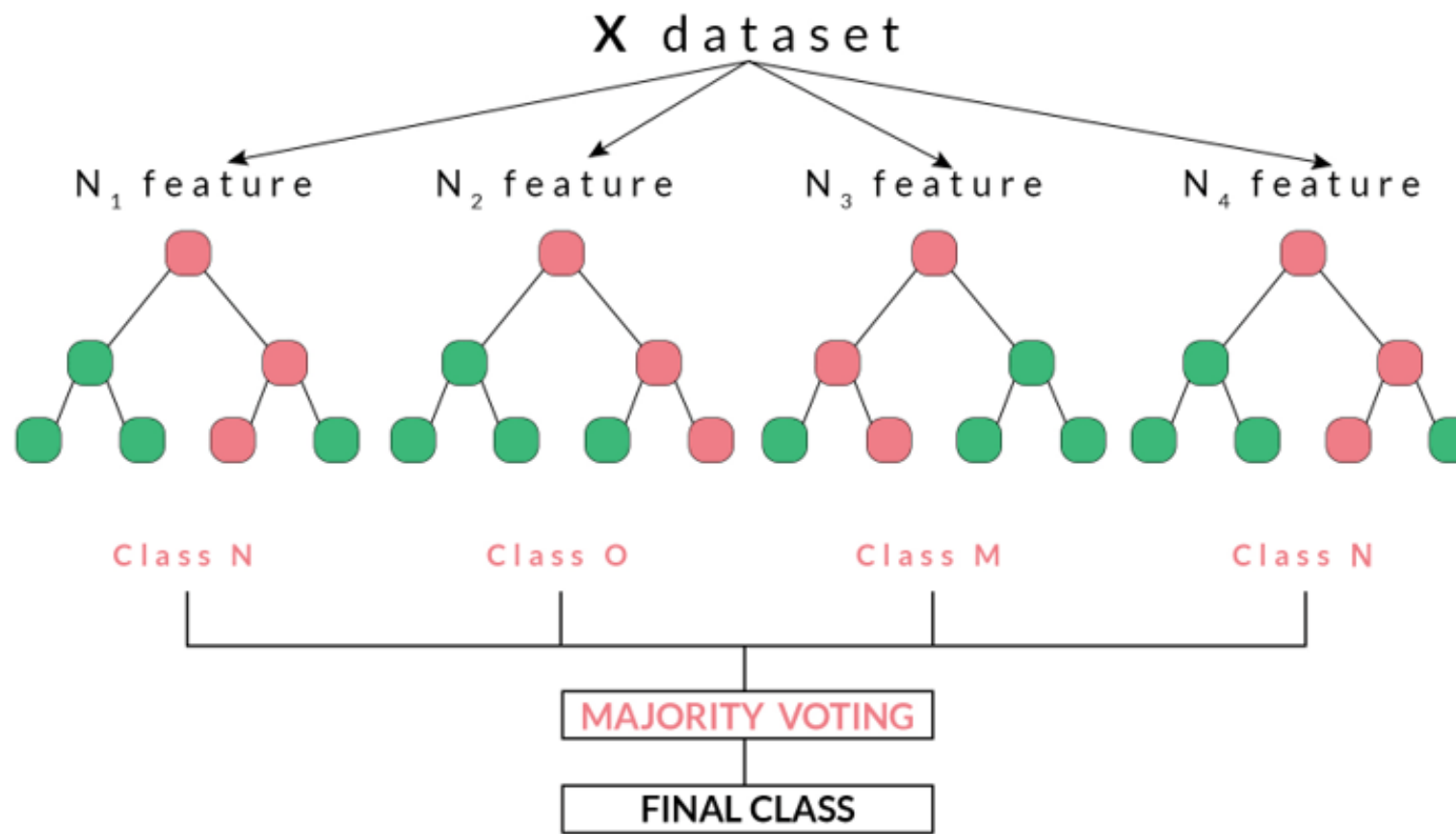


01 Random Forest y Bagging

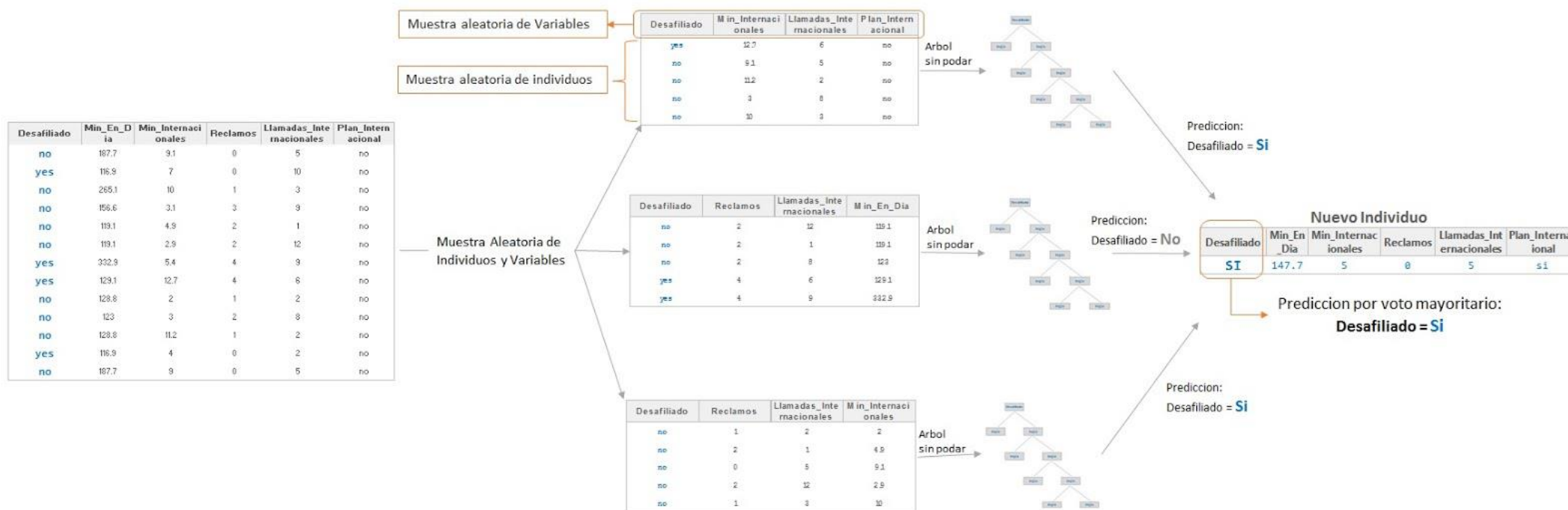
- Random forest es una clase de modelos ensamblados de tipo supervisado que se basa en un conjunto de árboles de decisión, donde cada uno de estos árboles es entrenado en una submuestra aleatoria (con reemplazo).
- El punto clave de random forest es utilizar una serie de árboles, cada uno entrenado con diferentes registros y también con diferentes atributos, lo que le permite mejorar sus predicciones.



01 Random Forest y Bagging



01 Random Forest y Bagging



Laboratorio

06

05 Ejercicio Taller

Esta parte tiene el objetivo poner en práctica todo lo visto durante esta sección, con un ejercicio práctico, que describiremos a continuación:

Utilizaremos los datos que han sido generados por el Cherenkov Gamma Telescope, que es un telescopio que en base a imágenes es capaz de registrar partículas gamma de alta energía. Para ello, el set de datos nos entrega 10 atributos de la imagen, dentro de los que se encuentran medidas de la elipse, conteo de pixeles, ángulos, distancias, y la variable target que tiene dos clases: g para gamma (señal bien clasificada) o h para hadrón (señal no capturada).

El set de datos cuenta con 19.020 registros, y se busca un modelo de clasificación para determinar futuras señales bien capturadas.

05 Ejercicio Taller

Usted debe hacer lo siguiente:

1. Hacer una exploración de los datos y decidir qué modelo va a utilizar. Fundamentar su elección.
2. En base a su elección anterior, procese sus datos para dejarlos como input para el modelo, incluyendo los pasos necesarios del machine learning pipeline.
3. Una vez ajustado su modelo, entregue métricas de desempeño, y argumente si es necesario o no realizar una optimización mediante hiper-parámetros. Si es el caso, hágalo
4. Reporte resultados finales y coméntelos.

SMART +
SUSTAINABLE

Liderar la construcción **de un mundo sostenible**

Aprendizaje Supervisado

Bastián Galasso

Data Analytics Lead – Coca-Cola Embonor

Julio 2025